Skills as Files — versioned markdown prompts as code artifacts

SPEC-TH-031 v0.1 DRAFT Jeremie Nunez

1 Context

Thalamus cortices are driven by LLM system prompts. Embedding those prompts in TypeScript string literals makes them invisible to non-engineers, impossible to diff meaningfully, and awkward to review. This spec promotes prompts to first-class, versioned artifacts: markdown files under `packages/thalamus/src/cortices/skills/`, loaded at runtime by `cortex-llm.ts`, validated in CI, and reviewable by analysts and compliance without touching TypeScript.

2 Goals

- Every cortex prompt lives in a dedicated `.md` file next to code.
- Each skill file carries structured front-matter: `name`, `description`, `sqlHelper`, `params`.
- The body declares *purpose*, *inputs*, *outputs*, and *examples* in fixed sections.
- CI validates the format so a malformed skill cannot reach `main`.
- Non-engineers can open a PR editing only markdown and ship a behavior change safely.

3 Non-Goals

Non-Goal

This spec does not define the DAG execution order, the transport to the LLM provider, or the structure of the knowledge graph writes (covered in SPEC-TH-030). It does not replace secret configuration — API keys remain in the environment.

4 Invariants

Invariant

A skill file is the single source of truth for its cortex's prompt. The TypeScript code MUST load the file at runtime (or at build time via a typed registry) and MUST NOT duplicate the prompt as a string literal.

Invariant

Every skill file satisfies the schema: YAML front-matter with `name, description, sqlHelper, params`, followed by a body that contains at least the sections `Purpose`, `Inputs`, `Output Format`, and `Examples`. Any deviation fails CI.

Invariant

The `Output Format` section declares a JSON shape. The runtime parses LLM responses against that shape; rejections are logged with the skill name and a sample. No silent coercion.

5 Interface

File layout — a skill is a single markdown document:

```
---
name: strategist              # required, kebab-case, matches filename
description: >                # required, one-paragraph summary
  Meta-synthesis cortex that reads findings from upstream cortices
  and produces portfolio-level recommendations.
sqlHelper: none               # required, a registered helper or "none"
params:                       # required, may be empty
  maxFindings: number
  lang: "fr" | "en"
---

# Strategist

## Purpose
One paragraph. What does this cortex decide, and why does it exist.

## Inputs
Bulleted list of what arrives in the DATA payload.

## Output Format
Return JSON: `{ "findings": [...] }`. Each finding has fields
title, summary, findingType, confidence, impactScore, evidence, edges.

## Examples
At least one worked example: input shape -> expected finding.
```

Loader API — consumed by `cortex-llm.ts`:

```typescript
// packages/thalamus/src/cortices/skills/registry.ts

export interface SkillFile {
  name: string;
  description: string;
  sqlHelper: string;          // key into sql-helpers registry, or "none
    "
  params: Record<string, string>;
  systemPrompt: string;       // the body, minus front-matter
  sourcePath: string;         // for logs and traceability
  sha256: string;             // content hash, stamped into provenance
}

export function loadSkill(name: string): SkillFile;
export function listSkills(): SkillFile[];
export function validateSkill(raw: string): {
  ok: boolean; errors: string[];
};
```

6 Scenarios

Scenario: Analyst edits a skill

Given an analyst updates the `Output Format` section of `strategist.md`
When the PR is opened
Then CI parses the front-matter and validates the required sections
And the diff is human-readable (markdown, not escaped strings)
And no TypeScript file needs to change for the update to ship.

Scenario: Missing required section fails CI

Given a skill file lacks an `Output Format` section
When `validateSkill` runs in CI
Then the build fails
And the error message names the file and the missing section.

Scenario: Filename and `name` must match

Given a file `strategist.md` declares `name: strategic-synth` in front-matter
When the validator runs
Then the build fails with `skill name must equal filename stem`.

Scenario: Runtime loads the file, not a literal

Given `cortex-llm.ts` needs the strategist prompt
When `analyzeCortexData` is invoked
Then the system prompt is loaded via `loadSkill("strategist")`
And the finding's provenance records the skill's `sha256`.

Scenario: Prompt hash reaches the knowledge graph

Given a cortex writes a finding to the graph (see SPEC-TH-030)
When the write lands
Then the row's provenance includes the `skillSha256` that produced it
And a later audit can diff the exact prompt version used at write time.

7 Acceptance Criteria

Acceptance Criterion AC-1

`validateSkill` applied to every file under `packages/thalamus/src/cortices/skills/` returns `ok: true` on a clean tree.

Acceptance Criterion AC-2

Seeding a malformed skill (missing `Output Format`) causes CI to fail with a message naming the file and the missing section.

Acceptance Criterion AC-3

`listSkills()` returns exactly the set of files in the skills directory; renaming a file and not updating `name` in front-matter fails validation.

Acceptance Criterion AC-4

No TypeScript file under `packages/thalamus/src/cortices/` contains a multi-line string literal longer than 200 characters that looks like a system prompt; a lint rule enforces this.

Acceptance Criterion AC-5

The `sha256` of each loaded skill is deterministic across machines (normalized line endings) and is included in provenance records.

Acceptance Criterion AC-6

An analyst-only PR (markdown-only diff) passes CI and deploys without requiring engineering sign-off beyond normal review.

Acceptance Criterion AC-7

When the LLM response fails to match the declared `Output Format`, the error log includes `skillName`, `skillSha256`, and a truncated sample of the response.

8 Traceability

AC	Test file	Test name
AC-1	`packages/thalamus/tests/skills/validate.spec.ts`	`all shipped skills are valid`
AC-2	`packages/thalamus/tests/skills/validate.spec.ts`	`missing Output Format fails`
AC-3	`packages/thalamus/tests/skills/registry.spec.ts`	`filename must match name`
AC-4	`packages/thalamus/tests/lint/no-inline-prompts.spec.ts`	`no long prompt literals`
AC-5	`packages/thalamus/tests/skills/registry.spec.ts`	`sha256 is deterministic`
AC-6	`packages/thalamus/tests/ci/analyst-pr.spec.ts`	`markdown-only PR passes`
AC-7	`packages/thalamus/tests/cortex-llm/output-format.spec.ts`	`mismatch is logged with sha`

9 Open Questions

- Should `params` use a JSON Schema or a lightweight DSL? JSON Schema is standard but verbose in front-matter.
- Do we version skills by semver in the front-matter as well as by git history?
- Should the `Output Format` block be executable (e.g. an embedded Zod schema) or remain prose plus a documented JSON shape?
- Localization — one file per language, or a single file with tagged sections?